

TOWARD ACCURATE SPEAKER INVENTORIES FOR ONLINE SPEAKER DIARIZATION

Youngki Kwon Hee-Soo Heo Minjae Lee Bong-Jin Lee

NAVER Cloud Corporation, South Korea

ABSTRACT

An online speaker diarizer maintains a growing inventory of speakers, and applications read it directly: a live transcript enumerates them as they appear. Existing systems either fix the inventory’s size in advance, so speakers beyond that size are merged, or grow it on distance alone, so noise and turn boundaries split speakers. Scored on speaker count over nine public datasets, both failures appear where DER reports neither: on VoxSRC-23 the fixed-capacity systems return 44–46% too few speakers and the unbounded tracker with the lowest DER 63% too many. We build on that embedding-based tracker and change only what triggers a registration. A candidate speaker pool keeps unregistered embeddings apart and registers a speaker only once one candidate has gathered enough mutually compatible embeddings; commit-gated label assignment holds an output while its candidate is undecided, so no speaker appears in the output before it is confirmed. Across the nine datasets the macro speaker-count error falls from 225% to 21% of the reference and macro DER from 17.59% to 15.95%, at a mean added latency of 0.109 s over all outputs; on VoxSRC-23 the tracker obtains both the lowest DER and the most accurate speaker count of the systems compared.

Index Terms— online speaker diarization, speaker inventory, speaker counting, speaker registration, latency

1. INTRODUCTION

Speaker diarization determines “who spoke when” [1, 2]. An online system must emit temporally ordered labels under bounded latency.

Every online speaker diarizer maintains a growing set of speakers admitted so far, which applications read directly; we call it the *speaker inventory*—built online, not enrolled in advance. Prior work has no shared name for it—it appears as attractors [3, 4], as a speaker cache [5], as a speaker-tracing buffer [6, 7], as centroids or retained samples [8–11]—and the designs differ mainly in how they decide when to add to it.

Two kinds of rule govern how that set grows. Some designs fix its size as an architectural constant settled before the session—a *speaker capacity*, the most speakers the system can ever return: a number of output slots [5] or an attractor budget set by training [4]; block-wise designs [3, 6, 7] apply that budget within each block rather than to the whole recording. Embedding-based designs [8–11] leave it open and decide online: each incoming embedding is compared against the inventory already held, and a speaker is admitted when nothing there is close enough. Either way the rule is fixed in advance, and once a label is emitted it cannot be revised, so a registration taken on too little evidence is never undone.

An inventory has a *count error* when it returns a different number of speakers than are present, in one of two directions. Too few is a merge: two or more speakers share one identity in the inventory, and the speech that cannot be placed is charged to speaker confusion—what a fixed speaker capacity produces once the true count passes it. Too many is a split: one speaker is spread over several identities—what unbounded

registration produces, since every sufficiently distant embedding becomes a speaker, and conversation supplies them constantly. We measure it simply, as the total absolute difference in speaker count over the total reference count, both directions counted (Section 4).

DER, the primary metric of speaker diarization [1], reports neither direction: it weights speakers by the time they hold and does not state how many came back. Among the existing systems on VoxSRC-23 (Table 1), the lowest DER comes with the worst inventory, and the most accurate inventory with more than twice that DER. JER [12], another notable metric, weights every reference speaker equally, which raises the weight of short speakers, but it is still scored over the optimal speaker mapping and is silent on the count and its direction.

We follow the embedding-based designs, since only an inventory with no fixed speaker capacity can hold the many speakers our target recordings contain. We build on DC-OST [13], keeping its routing rule and its delayed commitment (Section 2). What we change is what triggers a speaker registration (Section 3). We compare against existing systems on DIHARD III and VoxSRC-23, re-running the released ones under the same protocol and scoring DER and speaker-count error together. The proposed tracker is then analysed on nine datasets, component by component and over its hyper-parameters, with the latency it costs (Section 5).

Our contributions are:

- A *candidate speaker pool* that decides when the inventory grows (Section 3.1). A new speaker is registered only on accumulated evidence, kept separate per candidate. The number of speakers registered therefore stays close to the number present: macro speaker-count error falls from 225% to 21% of the reference, and macro DER by 0.77 %p.
- *Commit-gated label assignment* over it (Section 3.2). An output that may belong to a new speaker waits until the next few outputs confirm or reject that speaker. Look-ahead is therefore spent only where needed, not fixed on every output: macro DER falls a further 0.87 %p, at a mean added 0.109 s over all outputs—1.27 s over the 8.6% held.

2. BACKGROUND

Our baseline is DC-OST [13], an embedding-based tracker with no fixed speaker capacity. Throughout, t indexes the embedding steps, taken every 0.5 s over detected speech only, so consecutive steps can be far apart in time; a_t is the audio time of step t and $\mathbf{e}_t$ the embedding of the window ending there.

The tracker holds a registry of committed centroids, $\mathcal{S}_t = \{\mathbf{c}_{t,i}\}_{i=1}^{N_t}$: $\mathbf{c}_{t,i}$ is the centroid of speaker i at step t , updated as the speaker accumulates observations, and N_t the number of registered speakers. It also holds a single buffer B of embeddings that matched none of them. It routes $\mathbf{e}_t$ by its nearest centroid under the cosine distance d : with $d_{\min} = \min_{1 \leq i \leq N_t} d(\mathbf{e}_t, \mathbf{c}_{t,i})$, the embedding is assigned to that speaker if $d_{\min} \leq \tau_t$ and appended to B otherwise, where τ_t grows linearly with N_t .

Algorithm 1 One embedding step of the proposed tracker. Lines 2–4 are the common case, line 3 the routing test; the two gates are the tests on τ_{pool} (line 7) and K (line 10).

Input: embedding $\mathbf{e}_t$ at audio time a_t
Output: the labels released at step t : zero or more identifiers, each for the output held at audio time a , for $\mathbf{e}_t$ or for earlier steps still in $\mathcal{Q}_t$
State: registry $\mathcal{S}_t$ with $N_t = |\mathcal{S}_t|$ speakers; pool $\mathcal{P}_t$ of candidates P_j , each a set of embeddings $\mathbf{x}$ with their audio times, $\bar{P}_j$ their mean, $\text{last}(P_j)$ the newest; pending queue $\mathcal{Q}_t$ of entries (a_t, i^*, P^*) : audio time, fallback label, candidate ($\emptyset$ once decided)
Hyper-parameters: admission threshold τ_{pool} , registration size K , delay budget L_{prov} , expiry T_e , and $\tau_{\text{base}}, \lambda, \tau_{\text{max}}$ of the adaptive routing threshold $\tau_t = \min(\tau_{\text{base}} + \lambda N_t, \tau_{\text{max}})$

- 1: **if** $N_t = 0$ **then** $N_t \leftarrow 1$; $\mathbf{c}_{t,1} \leftarrow \mathbf{e}_t$
- 2: $i^* \leftarrow \arg\min_{1 \leq i \leq N_t} d(\mathbf{e}_t, \mathbf{c}_{t,i})$; $d_{\min} \leftarrow d(\mathbf{e}_t, \mathbf{c}_{t,i^*})$
- 3: **if** $d_{\min} \leq \tau_t$ **then**
- 4: update $\mathbf{c}_{t,i^*}$ with $\mathbf{e}_t$; append $(a_t, i^*, \emptyset)$ to $\mathcal{Q}_t$
- 5: **else**
- 6: $D_j \leftarrow \text{median}(\{d(\mathbf{e}_t, \mathbf{x}) : \mathbf{x} \in P_j\})$ for each $P_j \in \mathcal{P}_t$
- 7: **if** $\min_j D_j \leq \tau_{\text{pool}}$ **then** $P^* \leftarrow \arg\min_j D_j$; $P^* \leftarrow P^* \cup \{\mathbf{e}_t\}$
- 8: **else** $P^* \leftarrow \{\mathbf{e}_t\}$; $\mathcal{P}_t \leftarrow \mathcal{P}_t \cup \{P^*\}$
- 9: append (a_t, i^*, P^*) to $\mathcal{Q}_t$
- 10: **if** $|P^*| \geq K$ **then**
- 11: $N_t \leftarrow N_t + 1$; $\text{id}(P^*) \leftarrow N_t$
- 12: $\mathbf{c}_{t,N_t} \leftarrow \bar{P}^* / \|\bar{P}^*\|$; $\mathcal{P}_t \leftarrow \mathcal{P}_t \setminus P^*$
- 13: **end if**
- 14: **end if**
- 15: **while** $\mathcal{Q}_t \neq \emptyset$ and its head (a, i, P) has $P = \emptyset$, P registered, or $a_t - a \geq L_{\text{prov}}$ **do**
- 16: pop it; emit $\text{id}(P)$ if P registered, else i
- 17: **end while**
- 18: discard every $P_j \in \mathcal{P}_t$ with $|P_j| = 1$ and $a_t - \text{last}(P_j) \geq T_e$

A label is emitted at once—the matched speaker’s, or the next unused identifier $N_t + 1$ if $\mathbf{e}_t$ went to the buffer—and a speaker is registered only when $|B|$ reaches a preset size K , with the medoid of B as its centroid.

Limitations. Two properties of that procedure bound it. (1) One buffer for every unseen speaker: two unseen speakers who alternate mix in B , share one identifier, and are registered as one speaker with the medoid of the mixture as its centroid. (2) Every decision from one observation: a single embedding with $d_{\min} > \tau_t$ starts a new speaker, labelled $N_t + 1$ at once, so one outlier can seed a speaker who does not exist.

3. PROPOSED METHOD

Section 2 ends on two limitations; the proposed tracker answers each. The single buffer becomes a pool $\mathcal{P}_t$ of candidate speakers kept apart. And no decision rests on one observation. An embedding is still placed at once, into a candidate, but the candidate becomes a speaker only after enough mutually compatible embeddings have joined it. An output whose candidate is undecided is held in an ordered queue $\mathcal{Q}_t$ until then: DC-OST delays the registration, we delay the label as well. Nothing else in the pipeline changes. Algorithm 1 states one embedding step in full; the rest of this section gives the reasons behind its two gates.

3.1. Evidence-gated speaker registration

The pool $\mathcal{P}_t$ replaces B with candidate speakers that stay apart, the same K now applying to each candidate separately rather than to one shared buffer. An embedding the registry rejects is scored against each candidate by the median of its member-wise distances (line 6). It joins

the nearest candidate if that distance is within the admission threshold τ_{pool} , and opens a new one otherwise (lines 7–8). Two unseen speakers who alternate therefore accumulate in separate candidates instead of one buffer.

A candidate is registered once it holds K members (line 10). Because every member joined by passing the admission test of line 7, the K embeddings are accumulated evidence for one speaker rather than K embeddings that happened to miss the registry. The committed centroid is the normalized mean of those members (line 12). A candidate below K receives no label at all, so a single noisy observation leaves no trace in the inventory, and beyond the first speaker (line 1) no speaker is registered until one candidate holds K of its embeddings; a candidate with $|P_j| = 1$ after T_e of audio is discarded (line 18).

3.2. Commit-gated label assignment

An output that joins an unresolved candidate is queued with both identities it might take (line 9): a *fallback*, the nearest speaker already in $\mathcal{S}_t$, and the *provisional* identity of its candidate. Whether that candidate becomes a speaker is settled within a few steps, so the queue holds the output instead of guessing. The wait is timed out at L_{prov} , the delay budget, checked at each embedding step after registrations are updated: the output leaves under the provisional identity as soon as its candidate is registered, and under the fallback once its wait reaches L_{prov} with the candidate still unregistered (lines 15–16). No speaker therefore appears in the output before it is confirmed. A timed-out output is not revisited; if its candidate registers later, the error is a merge. The queue releases in step order. A routed output is decided on arrival (line 4), so it waits only behind an earlier undecided one, never for evidence of its own. We call that wait *head-of-line* blocking, as in queueing: an output held back only by the one in front of it. L_{prov} therefore applies to the queue head, not to the queue.

4. EXPERIMENTAL SETUP

4.1. Datasets

We evaluate on nine public datasets, 1,907 recordings. *DIHARD I–III* [12, 14, 15] span eleven domains and are the hardest acoustic conditions of the nine: 1 to 9 speakers per recording, most of them under four, with moderate overlap. *VoxConverse* [16] and *VoxSRC 2021–2023* [17] are in-the-wild video, and carry the many-speaker regime this paper is about: a wide range of speakers, from 1 to 28, with the least overlap of the nine. *AMI* [18] and *AliMeeting* [19] are English and Mandarin meetings—four speakers in all but one AMI recording, two to four in AliMeeting—and carry the most overlap. The comparison with existing systems (Table 1) uses the evaluation sets of the two challenge series, VoxSRC-23 and DIHARD III; our three variants are compared on all nine.

4.2. Evaluation protocol

We report DER with its components—false alarm (FA), missed speech (MS) and speaker confusion (SC)—in percent, with differences in percentage points (%p), and latency as a distribution (Section 5.4).

Overlap floor. Our tracker emits one label per step and misses overlapped speech; the existing systems are scored on their own overlap-aware output. We therefore report each dataset’s single-label missed-speech floor, below which our MS cannot fall.

Evaluation collar. Each dataset is scored under its own protocol—a 0.25-s collar on the Vox datasets, none elsewhere—so the per-dataset columns stay comparable to published numbers. A macro average over the nine would then mix two protocols, so every macro average we report is computed after rescaling all nine under one common 0.15-s collar.

Table 1. Comparison with existing systems (settings in Section 4). Rows other than Baseline rule and + Pool + Delay are our runs of the released artifacts, except S2SND-M[†], which is quoted; published DIHARD III DERs: DIART 27.6, Streaming Sortformer 18.97, LS-EEND 19.61. Lat. = algorithmic latency (s); δ is the queue’s added delay: zero for 91.4% of outputs, 1.27 s on average for the 8.6% that wait (0.109 s over all outputs), observed maximum 4.16 s (Section 5.4, Table 4). Bold = lowest DER and $|e|$ within each block.

System	FA	MS	SC	DER	$ e $	Lat.
<i>VoxSRC-23 (≤ 28 spk, MISS floor 1.6%)</i>						
DIART [8]	2.96	3.06	11.45	17.46	27.7	5.00
Streaming Sortformer [5]	1.10	4.23	15.45	20.79	46.2	1.04
LS-EEND [4]	2.62	8.40	21.49	32.51	43.7	1.00
Baseline rule	1.87	2.48	3.05	7.40	63.1	0.50
+ Pool + Delay	1.87	2.48	2.41	6.76	16.0	0.50+ δ
<i>DIHARD III (MISS floor 9.5%)</i>						
DIART [8]	5.83	9.65	13.06	28.53	27.0	5.00
Streaming Sortformer [5]	6.15	9.68	3.93	19.76	18.7	1.04
LS-EEND [4]	6.13	9.77	5.49	21.39	14.5	1.00
S2SND-M [†] [23]	—	—	—	17.12	—	0.80
Baseline rule	3.98	13.41	10.89	28.28	294.6	0.50
+ Pool + Delay	3.98	13.41	6.92	24.31	23.2	0.50+ δ

Latency. All latencies are algorithmic, not wall-clock. An output is one 0.5-s embedding step’s label, and its *added* latency is how long the pending queue holds it past the step at which the Baseline rule of Section 4.4 would have emitted it.

Speaker count error. For recording i , let N_i^{sys} be the number of distinct speakers the system returns—a label counts if it is emitted even once—and N_i^{ref} the number in the reference. The signed error is $\varepsilon_i = N_i^{\text{sys}} - N_i^{\text{ref}}$: a merge leaves a *deficit* ($\varepsilon_i < 0$) and a split an *excess* ($\varepsilon_i > 0$). We report $|e| = \sum_i |\varepsilon_i| / \sum_i N_i^{\text{ref}}$ in percent, which counts both alike instead of letting them cancel. It measures the count, not which identities are right; DER covers the time mislabelled.

4.3. Implementation details

Beyond two detector operating points, nothing below varies across the nine datasets.

Front-end module. The pipeline uses the public FireRed-VAD streaming detector [20]—a 0.6M-parameter causal DFSMN with zero look-ahead—and the released ReDimNet2 checkpoint b6-vb2+vox2.v0-1m.pt [21, 22], trained on VoxBlink2 and VoxCeleb2, neither of them among the nine evaluation datasets; embeddings from a 1.5-s window every 0.5 s give a base latency of 0.5 s. The detector runs at one operating point per protocol family, chosen on the DIHARD III and VoxConverse development sets: trimmed segment ends where the collar is zero, padded ones under the 0.25-s collar. No evaluation dataset appears in the detector’s training description.

Tracker. We use $K = 5$, $\tau_{\text{pool}} = 0.45$, $L_{\text{prov}} = 2.0$ s, singleton expiry $T_e = 30$ s (untuned), and, for the adaptive routing threshold τ_t shared with the baseline rule, $\tau_t = \min(\tau_{\text{base}} + \lambda N_t, \tau_{\text{max}})$ with $(\tau_{\text{base}}, \lambda, \tau_{\text{max}}) = (0.45, 0.005, 0.8)$ —all fixed on the DIHARD III and VoxConverse development sets as a trade-off between DER and $|e|$; no evaluation set enters any of these choices. The adaptive threshold is DC-OST’s mechanism [13], not one this paper introduces, so the sensitivity in Section 5.3 covers τ_{pool} and K ; L_{prov} is swept in Section 5.4.

4.4. Systems compared

We compare three variants of our own, three released systems run at their published settings, and one we quote.

Our variants. The three share the front end and segmentation and differ in decision policy: **Baseline rule**, i.e. the routing rule of Sec-

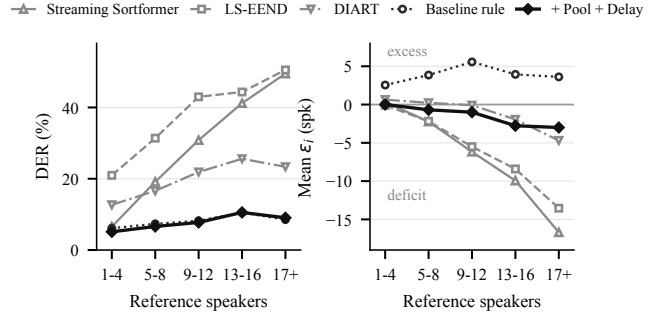


Fig. 1. Per-speaker-count behaviour on VoxSRC-23, grouped by the number of reference speakers (197/128/58/17/13 recordings). Left: DER. Right: mean signed count error ε_i (Section 4), speakers returned minus speakers present; zero means no net count bias.

tion 2 with one shared buffer at the same K [13]; **+ Pool** (Section 3.1); and **+ Delay** (Section 3.2). The Baseline rule registers the medoid of B as the centroid, the other two the normalized mean of a candidate’s members. Every variant labels every detected output—+ Pool releases an undecided output at once under its fallback ($L_{\text{prov}} = 0$), and the queue is flushed at the end of the stream—so FA and MS are identical across variants and every difference is speaker confusion.

Existing systems. They govern the inventory differently. **Streaming Sortformer** [5] has four output slots backed by an arrival-order cache, so a fifth speaker overwrites the oldest instead of receiving an identity; we run the released 4-speaker checkpoint [24] at its published 1.04-s setting. **LS-EEND** [4] fixes the attractor sequence to a constant length $S = 8$ and reads the active count below it from a termination marker; we use the official DIHARD III checkpoint [25] at 1 s, no Vox-tuned checkpoint being released. **DIART** [8] imposes no speaker capacity, holding one representation per speaker and creating a speaker when an embedding is farther than a constant from all of them; we run it at its default 5-s latency with the published hyper-parameters [26], the VoxConverse setting on VoxSRC-23 and the DIHARD III setting on DIHARD III. **S2SND-M** [23] releases no artifact, so we quote its published DIHARD III number.

5. RESULTS

5.1. Comparison with existing systems

On VoxSRC-23, + Pool + Delay has both the lowest DER and the lowest $|e|$ of the systems compared (Table 1); the Baseline rule is within 0.64 %p of its DER but returns 63.1% too many speakers. On DIHARD III our system trails Streaming Sortformer by 4.6 %p of DER—3.0 of confusion, 1.6 of FA + MS—and LS-EEND on $|e|$.

Fig. 1 splits VoxSRC-23 by reference speaker count. By DER the two embedding-based trackers, Baseline rule and + Pool + Delay, are indistinguishable in every group, while their inventories diverge from the first group on—the axis DER does not report. Streaming Sortformer returns at most four speakers, so beyond four its deficit is $N^{\text{ref}} - 4$ by construction, and its DER reaches 50% in the 17+ group. DIART carries no capacity and still ends short there. + Pool + Delay is near-exact up to four speakers and ends 3.0 short in the 17+ group, against 13.5 to 16.7 for the fixed-capacity systems.

None of the existing systems was trained on the VoxSRC-23 test data; DIART and LS-EEND run checkpoints from other domains there, while Streaming Sortformer’s training includes VoxConverse dev. On DIHARD III a fixed capacity helps when a recording has fewer speakers than it: both fixed-capacity systems count DIHARD III more accurately than our system does.

Table 2. Per-dataset results, both axes and both segmentation conditions. SC and DER as baseline rule / pool / pool with delay, bold = best; $|e|$ (%) as baseline rule / pool, the pool and the full tracker returning identical inventories. FA and MS are shared by the three variants; under reference boundaries FA is 0.00 throughout. Macro: all nine rescored under one 0.15-s collar (Section 4), so it is not the mean of the DER columns.

		detected speech					reference boundaries				
collar	Dataset	FA	MS	SC	DER	e	MS	SC	DER	e	
none	DIHARD I	5.31	13.58	12.58/9.16/ 8.55	31.47/28.05/ 27.45	278.4/21.4	8.71	15.17/10.30/ 9.60	23.88/19.01/ 18.32	329.2/24.4	
	DIHARD II	5.83	14.54	13.49/9.56/ 8.99	33.86/29.92/ 29.36	281.6/23.8	9.69	15.76/10.36/ 9.71	25.45/20.05/ 19.40	323.9/29.1	
	DIHARD III	3.98	13.41	10.89/7.34/ 6.92	28.28/24.73/ 24.31	294.6/23.2	9.52	12.31/7.42/ 6.92	21.83/16.94/ 16.44	331.9/25.2	
	AMI	1.95	26.32	6.13/3.66/ 3.46	34.40/31.93/ 31.74	466.7/47.6	14.55	7.44/4.59/ 4.35	21.99/19.14/ 18.90	460.3/50.8	
	AliMeet.	0.56	28.30	5.74 /6.00/5.90	34.60 /34.86/34.76	440.0/13.3	20.44	5.87/4.51/ 4.40	26.31/24.95/ 24.84	413.3/16.7	
0.25 s	VoxConv.	1.91	2.75	3.05/3.23/ 2.06	7.71/7.90/ 6.73	82.7/14.0	1.60	3.10/3.23/ 1.86	4.70/4.83/ 3.46	67.7/14.4	
	VoxSRC-21	1.45	2.30	3.08/3.95/ 2.42	6.83/7.70/ 6.16	59.5/12.1	1.69	3.22/3.95/ 2.21	4.91/5.63/ 3.89	48.4/11.4	
	VoxSRC-22	1.83	2.23	2.80/3.67/ 2.18	6.86/7.72/ 6.24	58.0/14.7	1.54	2.92/3.73/ 2.04	4.46/5.27/ 3.58	48.2/13.6	
	VoxSRC-23	1.87	2.48	3.05/3.78/ 2.41	7.40/8.14/ 6.76	63.1/16.0	1.64	3.22/3.83/ 2.28	4.86/5.47/ 3.92	54.3/16.0	
0.15 s Macro		1.99	9.66	5.95/5.17/ 4.31	17.59/16.82/ 15.95	225.0/20.7	6.08	6.62/5.26/ 4.27	12.71/11.35/ 10.35	230.8/22.4	

Table 3. Sensitivity to the two tracker hyper-parameters, macro DER and $|e|$ (%) under the common 0.15-s collar; * = the setting we run, bold = best in row. For K , DER is given with the budget fixed at $L_{\text{prov}} = 2.0$ s and matched to K , $(K-1) \times 0.5$ s; latency is the mean added latency of the matched run.

$\tau_{\text{pool}} (K=5)$	0.30	0.35	0.40	*0.45	0.50	0.55	0.60
DER	19.03	17.44	16.24	15.95	15.77	16.06	16.56
$ e $	25.1	21.5	20.3	20.7	25.9	37.2	55.6
$K (\tau_{\text{pool}}=0.45)$	3	4	*5	6	7		
DER, fixed	16.18	15.82	15.95	16.59	16.97		
DER, matched	16.18	15.82	15.95	16.44	16.70		
$ e $	63.6	29.3	20.7	19.5	20.5		
latency (s)	0.032	0.068	0.109	0.154	0.199		

5.2. Both axes across nine datasets

Table 2 presents the paper’s main result: on nine datasets both axes improve—the macro count error by an order of magnitude, DER on eight of the nine. The pool does most of the work wherever unseen speakers alternate in one buffer—confusion falls 3.55 %p on DIHARD III and 2.47 %p on AMI—and the delay adds little there. On the Vox datasets the ordering reverses: the pool alone *raises* confusion, by 0.18 to 0.87 %p, and the delay removes more than it added, 1.17 to 1.53 %p, because without the delay a genuinely new speaker’s first segments leave under a fallback label. AliMeeting is the one dataset where the pool does not help, and it carries the highest overlap floor: sustained two-speaker overlap yields recurring, mutually compatible *mixture* embeddings, the one input the evidence gate cannot reject. DER there is flat, 34.60 to 34.76, while $|e|$ goes 440% to 13%.

On the no-collar datasets 69–89% of the DER is FA and MS. To see the tracker without the detector’s errors we rescore with reference boundaries: there DER improves on all nine datasets, and the residual MS is exactly each dataset’s single-label floor. The count errors barely move: macro 225.0 to 230.8 and 20.7 to 22.4, the rises coming from the three DIHARD sets.

5.3. Sensitivity

Table 3 varies each hyper-parameter around the operating point of Section 4, one at a time over the nine-dataset macro average. Neither is best on both axes (bold), and what we run sits between them—0.18 and 0.13 %p off each sweep’s DER optimum, buying 5.2 and 8.6 %p of $|e|$ for it. Across both sweeps DER moves within 3.3 %p while $|e|$ swings from 19.5 to 63.6—which DER does not report—yet stays far below the baseline rule’s 225.0. The K block also gives DER with L_{prov} set to $(K-1) \times 0.5$ s instead of 2.0 s: $K=3$ and 4 are unchanged, and

Table 4. The delay budget, over all 1,473,619 outputs of the nine datasets. (a) L_{prov} sweep: macro SC (common 0.15-s collar) against the mean added latency over all outputs; * = the setting we run. (b) The 8.6% the queue holds, by exit (Section 3.2); % is of all outputs, mean and max added latency (s) are over the row.

(a) L_{prov} sweep			(b) the 8.6% that wait			
L_{prov} (s)	mean (s)	SC (%)	exit	%	mean	max
0.0	0.000	5.17	provisional	2.22	0.76	2.00
0.5	0.027	4.91	fallback	2.44	2.02	4.16
1.0	0.054	4.69	head-of-line	3.90	1.09	3.42
1.5	0.081	4.49				
*2.0	0.109	4.31	all	8.56	1.27	4.16
2.5	0.136	4.31				

$K=6$ and 7 recover 0.15 and 0.27 %p and still trail $K=5$.

5.4. The delay budget and what it costs

To place L_{prov} we sweep it, measuring macro SC against the mean added latency actually incurred (Table 4a); Table 4b splits the outputs the queue holds by how they leave. The sweep saturates at $(K-1)$ embedding steps—the time a candidate needs to gather its K members—exactly the $L_{\text{prov}} = 2.0$ s we adopt: a further 0.5 s buys nothing.

The front-end bounds are shared by every variant, so the tracker’s entire cost is the hold. It touches 8.6% of outputs, a mean added 0.109 s over all outputs and 1.27 s over those held (Table 4b): 2.2% resolve to their own candidate, 2.4% time out and leave under the fallback—the only guesses—and 3.9% were already decided and waited only behind an earlier undecided output, the head-of-line cost of releasing in step order. The 4.16-s maximum is a timeout that fell in a pause in speech, released at the next embedding step.

6. CONCLUSION

No system we measured keeps the speaker inventory accurate across speaker counts, and DER does not report it, so we report the speaker-count error alongside it. We kept the count open and let no decision rest on one observation: a candidate speaker pool registers a speaker only on accumulated evidence, and commit-gated label assignment holds an output while its candidate is undecided. Across nine datasets this cuts the macro speaker-count error from 225% to 21% and macro DER by 1.64 %p, at a mean added 0.109 s over all outputs; the count is settled by the pool alone, the delay only reallocates labels.

The clearest limitation is overlap: one label per step misses overlapped speech, four fifths of the meeting datasets’ residual after segmentation. Admitting more than one speaker per step is next.

7. REFERENCES

- [1] Xavier Anguera, Simon Bozonnet, Nicholas Evans, Corinne Fredouille, Gerald Friedland, and Oriol Vinyals, “Speaker diarization: A review of recent research,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 20, no. 2, pp. 356–370, 2012.
- [2] Tae Jin Park, Naoyuki Kanda, Dimitrios Dimitriadis, Kyu J Han, Shinji Watanabe, and Shrikanth Narayanan, “A review of speaker diarization: Recent advances with deep learning,” *Computer Speech & Language*, vol. 72, pp. 101317, 2022.
- [3] Shota Horiguchi, Shinji Watanabe, Paola García, Yuki Takashima, and Yohei Kawaguchi, “Online neural diarization of unlimited numbers of speakers using global and local attractors,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 706–720, 2023.
- [4] Di Liang and Xiaofei Li, “LS-EEND: Long-form streaming end-to-end neural diarization with online attractor extraction,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2025.
- [5] Ivan Medennikov, Taejin Park, Weiqing Wang, He Huang, Kunal Dhawan, Jinhan Wang, Jagadeesh Balam, and Boris Ginsburg, “Streaming Sortformer: Speaker cache-based online speaker diarization with arrival-time ordering,” in *Proc. Interspeech*, 2025, pp. 5238–5242.
- [6] Yawen Xue, Shota Horiguchi, Yusuke Fujita, Shinji Watanabe, Leibny Paola García, and Kenji Nagamatsu, “Online end-to-end neural diarization with speaker-tracing buffer,” in *Proc. SLT*, 2021, pp. 841–848.
- [7] Yawen Xue, Shota Horiguchi, Yusuke Fujita, Yuki Takashima, Shinji Watanabe, Leibny Paola García Perera, and Kenji Nagamatsu, “Online streaming end-to-end neural diarization handling overlapping speech and flexible numbers of speakers,” in *Proc. Interspeech*, 2021, pp. 3116–3120.
- [8] Juan M Coria, Hervé Bredin, Sahar Ghannay, and Sophie Rosset, “Overlap-aware low-latency online speaker diarization based on end-to-end local segmentation,” in *Proc. ASRU*, 2021, pp. 1139–1146.
- [9] Yucong Zhang, Qinqian Lin, Weiqing Wang, Lin Yang, Xuyang Wang, Junjie Wang, and Ming Li, “Low-latency online speaker diarization with graph-based label generation,” in *Proc. Speaker Odyssey*, 2022, pp. 162–169.
- [10] Yanyan Yue, Jun Du, Mao-Kui He, Yu Ting Yeung, and Renyu Wang, “Online speaker diarization with core samples selection,” in *Proc. Interspeech*, 2022, pp. 1466–1470.
- [11] Youngki Kwon, Hee-Soo Heo, Bong-Jin Lee, You Jin Kim, and Jee-weon Jung, “Absolute decision corrupts absolutely: conservative online speaker diarisation,” in *Proc. ICASSP*, 2023, pp. 1–5.
- [12] Neville Ryant, Kenneth Church, Christopher Cieri, Alejandrina Cristia, Jun Du, Sriram Ganapathy, and Mark Liberman, “The second DIHARD diarization challenge: Dataset, task, and baselines,” in *Proc. Interspeech*, 2019, pp. 978–982.
- [13] Youngki Kwon, Hee-Soo Heo, Minjae Lee, Han-Gyu Kim, and Bong-Jin Lee, “Delayed-commitment online speaker tracking for robust many-speaker diarization,” in *Proc. Interspeech*, 2026, to appear.
- [14] Neville Ryant, Kenneth Church, Christopher Cieri, Alejandrina Cristia, Jun Du, Sriram Ganapathy, and Mark Liberman, “First DIHARD challenge evaluation plan,” Tech. Rep., Linguistic Data Consortium, 2018.
- [15] Neville Ryant, Prachi Singh, Venkat Krishnamohan, Rajat Varma, Kenneth Church, Christopher Cieri, Jun Du, Sriram Ganapathy, and Mark Liberman, “The third DIHARD diarization challenge,” in *Proc. Interspeech*, 2021, pp. 3570–3574.
- [16] Joon Son Chung, Jaesung Huh, Arsha Nagrani, Triantafyllos Afouras, and Andrew Zisserman, “Spot the conversation: Speaker diarisation in the wild,” in *Proc. Interspeech*, 2020, pp. 299–303.
- [17] Jaesung Huh, Joon Son Chung, Arsha Nagrani, Andrew Brown, Jee-weon Jung, Daniel Garcia-Romero, and Andrew Zisserman, “The VoxCeleb speaker recognition challenge: A retrospective,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.
- [18] Jean Carletta, Simone Ashby, Sebastien Bourban, Mike Flynn, Mael Guillemot, Thomas Hain, Jaroslav Kadlec, Vasilis Karaiskos, Wessel Kraaij, Melissa Kronenthal, et al., “The AMI meeting corpus: A pre-announcement,” in *Proc. MLMI*, 2005, pp. 28–39.
- [19] Fan Yu, Shiliang Zhang, Yihui Fu, Lei Xie, Siqi Zheng, Zhihao Du, Weilong Huang, Pengcheng Guo, Zhijie Yan, Bin Ma, Xin Xu, and Hui Bu, “M2MeT: The ICASSP 2022 multi-channel multi-party meeting transcription challenge,” in *Proc. ICASSP*, 2022, pp. 9156–9160.
- [20] Kaituo Xu, Yan Jia, Kai Huang, Junjie Chen, Wenpeng Li, Kun Liu, Feng-Long Xie, Xu Tang, and Yao Hu, “FireRedASR2S: A state-of-the-art industrial-grade all-in-one automatic speech recognition system,” *arXiv preprint arXiv:2603.10420*, 2026.
- [21] Ivan Yakovlev and Anton Okhotnikov, “ReDimNet2: Scaling speaker verification via time-pooled dimension reshaping,” in *Proc. Interspeech*, 2026, to appear.
- [22] “ReDimNet2: released b6-vb2+vox2-v0-1m checkpoint,” github.com/PalabraAI/redimnet2.
- [23] Ming Cheng, Yuke Lin, and Ming Li, “Sequence-to-sequence neural diarization with automatic speaker detection and representation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2025.
- [24] “Streaming Sortformer: released 4-speaker checkpoint,” hf.co/nvidia/diar_streaming_sortformer_4spk-v2.
- [25] “LS-EEND: released implementation and DIHARD III checkpoint,” github.com/Audio-WestlakeU/FS-EEND/tree/main/LS-EEND.
- [26] “DIART: released implementation,” github.com/juanmc2005/diart.